A Dual-Network Feedback Communication System for Multi-Node Unmanned Robotic Control

Research Team — Affiliation

Draft — working paper

Abstract

This paper presents a dual-network communication architecture for a multi-node, remotely operated robotic platform built around a network of STM32 microcontrollers. Internal coordination between nodes is handled over an RS485 bus, while an external TCP/IP link, established through a Wiznet W5500 controller on the master node, carries control commands, multi-camera video, and real-time telemetry to a remote client application. The architecture cleanly separates deterministic, low-latency intra-robot messaging from scalable, location-independent external communication, and incorporates a bidirectional feedback path that returns positional, speed, torque, and status data from critical subsystems such as a six-degree-of-freedom (6DOF) manipulator arm and the drive system. We describe the system design, implementation, and a structured test program covering signal integrity, throughput, connection stability, latency, video access, and combined-network operation. The current platform is teleoperated rather than autonomous; however, the reliability of the communication and feedback layer is intended as the foundation on which higher-level autonomy — sensing, perception, and automated control — can later be built. We close by outlining a path toward that autonomy and the principal challenges involved.

1. Introduction

1.1. Background

Communication systems are a foundational element of any robotic platform. The ability to move commands, sensor readings, and status information reliably between distributed processing nodes — and between the robot and its operator — determines how responsive, scalable, and ultimately how autonomous a system can become. In multi-node robots, where separate microcontrollers govern distinct subsystems such as locomotion, manipulation, and sensing, the communication fabric is not a peripheral concern but a central design axis. A platform whose internal and external messaging is robust, low-latency, and observable provides exactly the substrate on which autonomous behaviours can later be layered, because autonomy ultimately depends on the same data paths that teleoperation relies on today.

This work focuses on a tracked robotic platform that integrates a multi-jointed manipulator, a drive system, and a set of networked cameras under coordinated remote control. The design deliberately treats communication as a first-class subsystem, separating the fast, deterministic exchanges that must happen inside the robot from the flexible, scalable exchanges that connect the robot to a remote operator.

1.2. Problem Statement

Remotely operated robots that aspire to eventual autonomy face a dual communication requirement that single-network designs struggle to satisfy. On one hand, internal coordination between subsystem controllers demands deterministic, low-latency, reliable messaging that is resilient to electrical noise in a motor-dense environment. On the other hand, the link to a remote operator must be scalable, support high-bandwidth video, and remain usable from arbitrary locations. Collapsing both requirements onto a single bus or a single network forces a compromise that degrades one role or the other.

The problem this paper addresses is therefore the design and validation of a communication and feedback architecture that meets both requirements simultaneously, while remaining simple enough to implement on low-cost microcontrollers and structured so that it can serve as the groundwork for autonomous operation in future development. The contribution is not a new autonomous algorithm but a reliable, observable, dual-network control and feedback layer — the layer on which such algorithms would depend.

2. System Overview

At a high level, the platform is organised around a single master controller and a set of slave controllers. The master is the only node that bridges the two networks: it participates in the internal RS485 bus alongside the slaves, and it alone holds the TCP/IP interface to the outside world. Every slave node owns a specific physical subsystem — an individual joint of the manipulator, a side of the drive system, or a pan-tilt-zoom (PTZ) camera mount — and reports back to the master on its own state.

This division yields three properties that the remainder of the paper develops. First, separation of concerns: time-critical motion coordination never competes with bulk video traffic, because the two live on physically different networks. Second, observability: because every slave reports structured telemetry to the master, and the master relays it to the client, the operator sees a continuously updated picture of the whole machine. Third, extensibility: adding a subsystem means adding a slave node and a message type, not redesigning the network. Together these properties are what make the architecture a credible starting point for autonomy, since an autonomous controller would consume exactly the same telemetry stream and emit exactly the same command set that the human operator does today.

3. System Design and Architecture

The system consists of multiple STM32 microcontrollers connected via RS485, with one acting as the master node. The master node is equipped with a Wiznet W5500 chip to handle TCP/IP communication, enabling remote control and feedback monitoring through a client software application running on a remote PC. The system also integrates six cameras, two of which are PTZ (Pan-Tilt-Zoom), providing video feed and control capabilities over the network. The dual-network approach is the defining feature of the design: RS485 handles intra-node communication while TCP/IP facilitates external communication.

3.1. RS485 Network

The RS485 network connects the master node to several slave nodes, each responsible for a different function, such as controlling the 6DOF arm, managing the drive system, or capturing sensor data. RS485 is well suited to this role because it is a differential, multi-drop bus: it tolerates electrical noise, supports

reliable communication over the distances involved inside the chassis, and allows many nodes to share a single pair of wires. The master disseminates commands to slaves predominantly by broadcast, with unicast used where a directed exchange is required. This network is essential for ensuring low-latency, reliable communication within the robot.

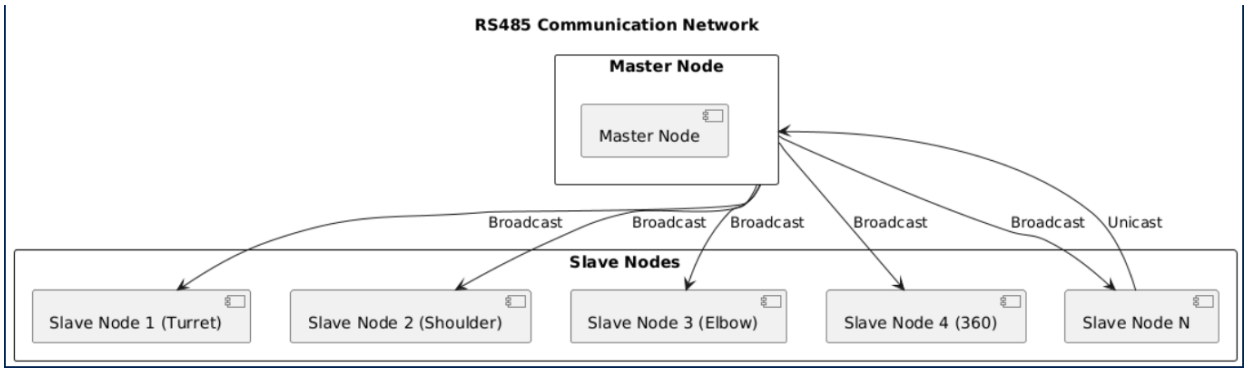

Figure 1. RS485 communication network. The master node broadcasts commands to the slave nodes (turret, shoulder, elbow, 360, and additional nodes) and receives directed (unicast) replies.

3.2. TCP/IP Network

The TCP/IP network, established via the Wiznet W5500 chip, connects the master node to the remote PC. This connection allows the client software to send control commands, access video feeds, and receive feedback from the robot. Cameras attach to the same network through a switch, and a wireless transceiver pair bridges the robot-side network to the remote operator. The use of TCP/IP ensures scalability and the ability to control the robot from virtually any location with network access.

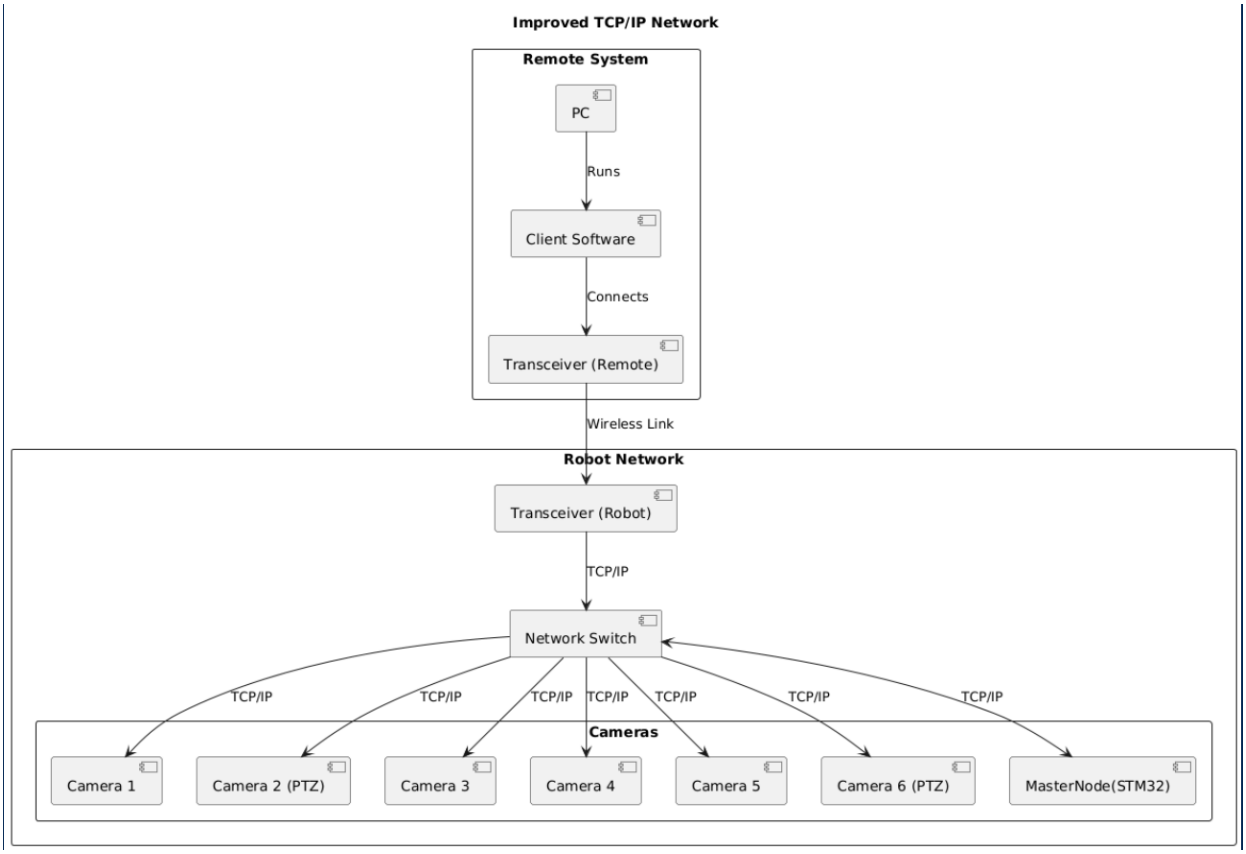

Figure 2. TCP/IP network. The remote client connects through a wireless transceiver link to the robot-side network switch, which serves the master node (STM32 + W5500) and the six cameras, two of them PTZ.

3.3. Overall Architecture

Figures 1 and 2 together illustrate the overall system architecture and the dual-network approach at its core. The master node is the single point of contact between the two networks: internally it is one peer on the RS485 bus, and externally it presents the TCP/IP endpoint that the client connects to. This means commands arriving over TCP/IP are translated by the master into RS485 traffic for the relevant slave, and telemetry collected over RS485 is aggregated by the master and relayed to the client. Keeping this bridging logic in one place simplifies both the firmware and the eventual insertion of an autonomous controller, which would attach at the same TCP/IP boundary.

4. Methodology

4.1. RS485 Network with Feedback Management

The RS485 network not only facilitates command dissemination from the master node to the slave nodes but also incorporates a feedback management system for critical components such as the 6DOF arm and the drive system. Each of these nodes is capable of sending real-time feedback to the master node, including data such as positional accuracy for the arm and speed, torque, and load conditions for the drive system.

This feedback loop is vital for ensuring that any deviation from expected performance can be corrected promptly, maintaining the integrity and responsiveness of the robotic system. The master node processes this feedback to make immediate adjustments where necessary and to log data for further analysis. In the current platform a human operator closes this loop through the client software; the same loop, however, is precisely what an autonomous controller would close automatically, which is why the feedback path is treated as a core part of the methodology rather than a monitoring convenience.

Each slave reports a small, fixed-format record to the master. For the arm joints these records carry an identifier, an operational status, speed, direction, an auxiliary data field, and a heartbeat counter; for the drive and PTZ subsystems the records carry the analogous motion and status fields. The heartbeat counter allows the master to detect a silent or failed node, and an explicit OFFLINE state distinguishes a disconnected subsystem from one that is merely idle.

4.2. Communication Testing via RS485

Test 1: Signal Integrity and Reliability

Setup. Connect the master node to multiple slave nodes via RS485.

Procedure:

- Send predefined command packets from the master node to each slave node.
- Measure the time taken for each node to receive and acknowledge the command.
- Record any instances of packet loss or corruption.

Expected Outcome. Reliable communication with minimal latency and no data loss or corruption.

Test 2: Data Throughput

Setup. Configure the system to continuously send and receive data packets of varying sizes.

Procedure:

- Measure the maximum data throughput between the master and slave nodes.
- Vary the baud rate to test its impact on throughput and reliability.

Expected Outcome. Throughput should align with RS485 capabilities at different baud rates without significant performance degradation.

4.3. TCP/IP Communication via Wiznet W5500

Test 3: TCP Connection Stability

Setup. Establish a TCP connection between the remote client software and the master node via the W5500 chip.

Procedure:

- Initiate a TCP connection and maintain it for an extended period (e.g., 24 hours).
- Monitor the connection for any drops or reconnections.

- Test the ability to reconnect automatically in case of a drop.

Expected Outcome. Stable TCP connection without drops, with quick reconnection if any interruption occurs.

Test 4: Latency and Command Response

Setup. Send control commands from the client software to the master node via TCP/IP.

Procedure:

- Measure the round-trip time (RTT) for commands sent from the client to the master node and back.
- Test with different network conditions (e.g., varying bandwidth, simulated packet loss).

Expected Outcome. Low-latency communication under normal network conditions, with acceptable performance under degraded conditions.

Test 5: Video Feed Access and Control

Setup. Access video feeds from the cameras via TCP/IP using the client software.

Procedure:

- Stream video from all cameras and monitor for frame drops or latency.
- Test PTZ control for the relevant cameras and measure response time.

Expected Outcome. Smooth video feed with minimal latency, and responsive PTZ controls.

4.4. Integration Testing

Test 6: Simultaneous RS485 and TCP/IP Communication

Setup. Operate both RS485 and TCP/IP networks simultaneously.

Procedure:

- Send commands via RS485 and TCP/IP concurrently.
- Measure any impact on performance or latency in either network.
- Monitor for any interference or communication failures.

Expected Outcome. Both communication networks operate efficiently without negatively impacting each other.

Test 7: Remote Control and Feedback Loop

Setup. Fully control the robot using the client software, sending commands via TCP/IP and receiving feedback through RS485.

Procedure:

- Perform various operations such as moving the robot, operating the 6DOF arm, and adjusting the PTZ cameras.

- Ensure that feedback from the robot (e.g., position, status) is accurately received and displayed on the client software.

Expected Outcome. Reliable remote control with real-time feedback, demonstrating the system's readiness for future autonomy.

5. Implementation

5.1. Hardware Components

The platform is realised on a tracked chassis that houses the drive system, power electronics, and the distributed controller network. The principal hardware components are listed below.

- STM32 microcontroller acting as the master node, hosting the RS485 interface and the W5500 TCP/IP interface.
- Multiple STM32 microcontrollers acting as slave nodes, one per subsystem (manipulator joints, drive sides, PTZ mounts).
- RS485 transceivers providing the differential, multi-drop bus that links the nodes.
- Wiznet W5500 controller providing hardware TCP/IP on the master node.
- Six cameras connected to the network, two of them PTZ, delivering video over TCP/IP.
- Remote PC running the client software, connected via TCP/IP through a wireless transceiver pair.

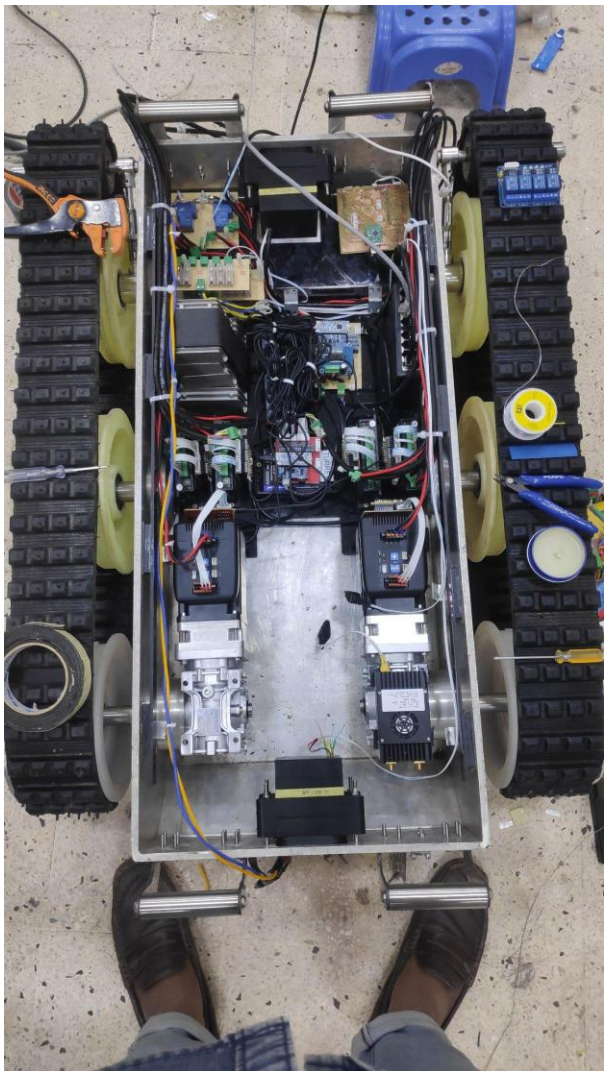

Figure 3. Assembled tracked chassis showing the drive motors, motor controllers, power electronics, and the distributed STM32 node network.

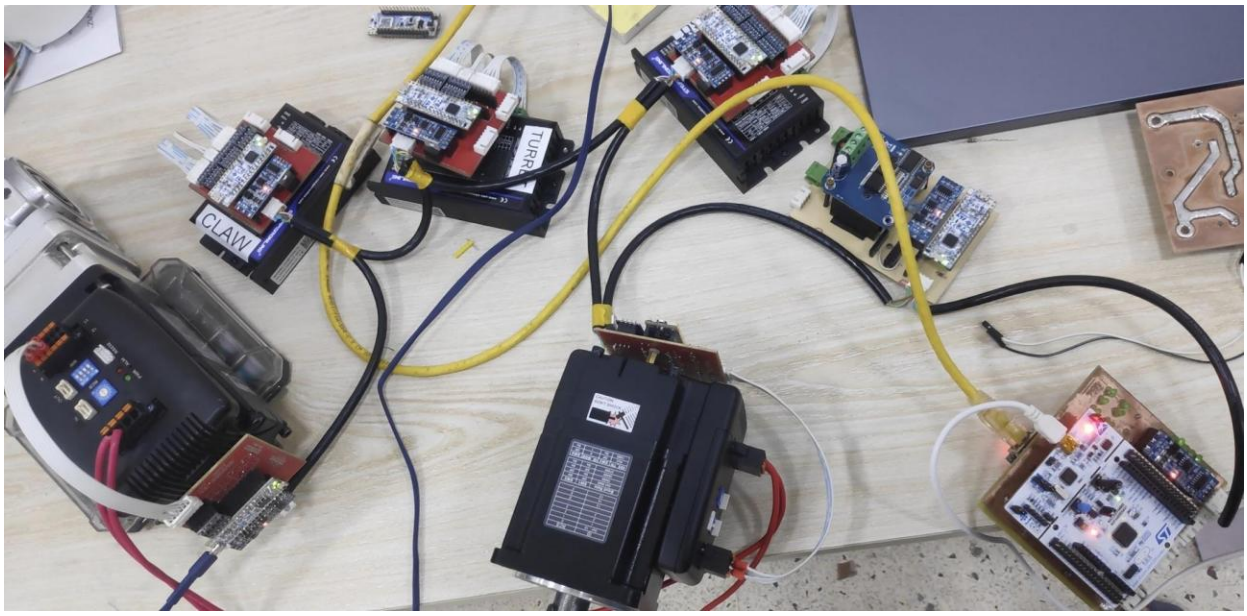

Figure 4. Bench integration of the subsystem nodes — labelled CLAW and TURRET controllers, motor drivers, a servo motor, and an STM32 development board — interconnected over RS485.

5.2. Software Components

The software comprises embedded firmware running on every node and a client application running on the remote PC.

- Embedded C/C++ firmware for the STM32 nodes, implementing the RS485 command and feedback protocol and, on the master, the bridging logic to TCP/IP.
- Client software for remote control and feedback monitoring, presenting per-subsystem telemetry and accepting operator commands.
- Configuration of the Wiznet W5500 for establishing and maintaining TCP connections to the client.

The client software surfaces the telemetry described in the methodology as live per-subsystem panels. Figure 5 shows the manipulator panel, with one tile per joint (turret, shoulder, elbow, wrist, 360, claw), each reporting identifier, status, speed, direction, an auxiliary field, and a heartbeat. Figure 6 shows the drive and PTZ panels, including an OFFLINE subsystem, which demonstrates the client's ability to distinguish a disconnected node from an idle one.

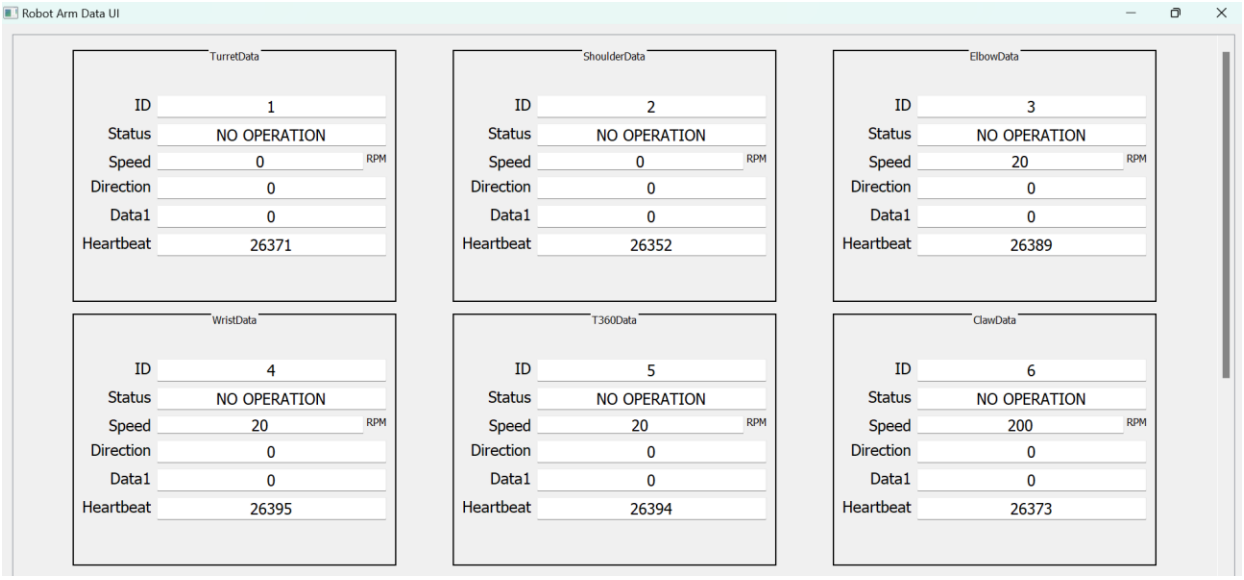

Figure 5. Client software — manipulator telemetry. Each tile corresponds to one joint of the 6DOF arm and reports live status, speed, direction, and a heartbeat counter.

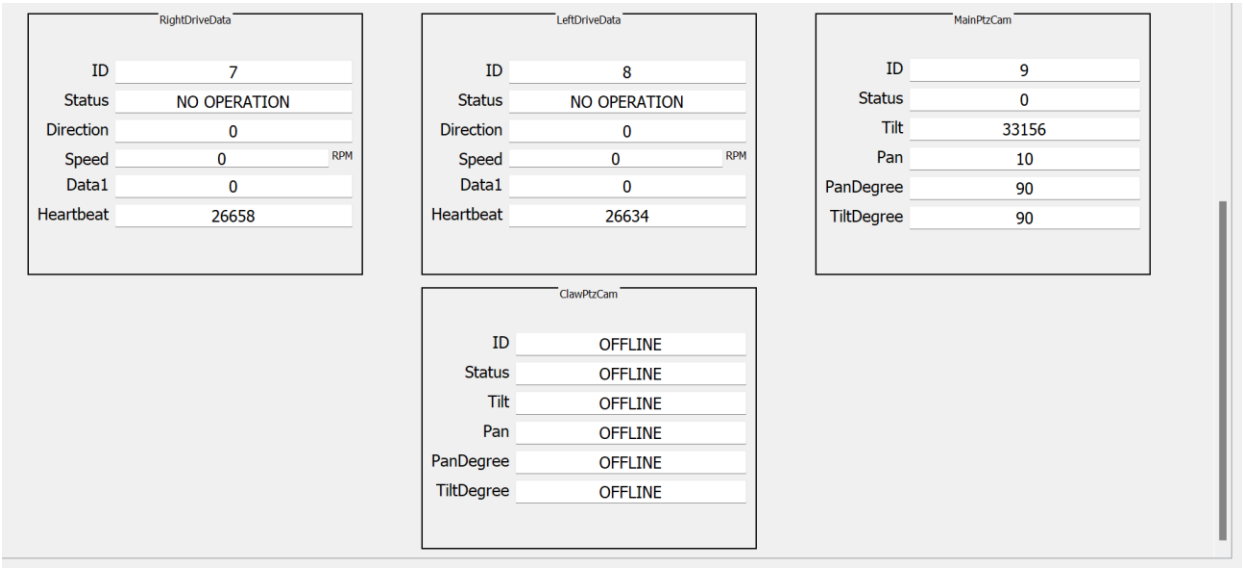

Figure 6. Client software — drive and PTZ camera telemetry. The left and right drive panels report speed and heartbeat; the PTZ panels report pan/tilt state, with one camera shown OFFLINE.

5.3. Communication Protocols

The choice of protocols reflects the dual requirement set out in the problem statement. RS485 was selected for the internal bus because its differential signalling is robust against the electrical noise generated by nearby motors and drivers, and because its multi-drop topology lets a single bus serve many nodes with minimal wiring. TCP/IP was selected for the external link because it provides reliable, ordered delivery, is natively supported by the W5500 in hardware, and makes the robot reachable from any networked location. The reliability and robustness of these protocols are what make them suitable not only for teleoperation but for autonomous operation, in which the same command and telemetry channels would be driven by software rather than by a human operator.

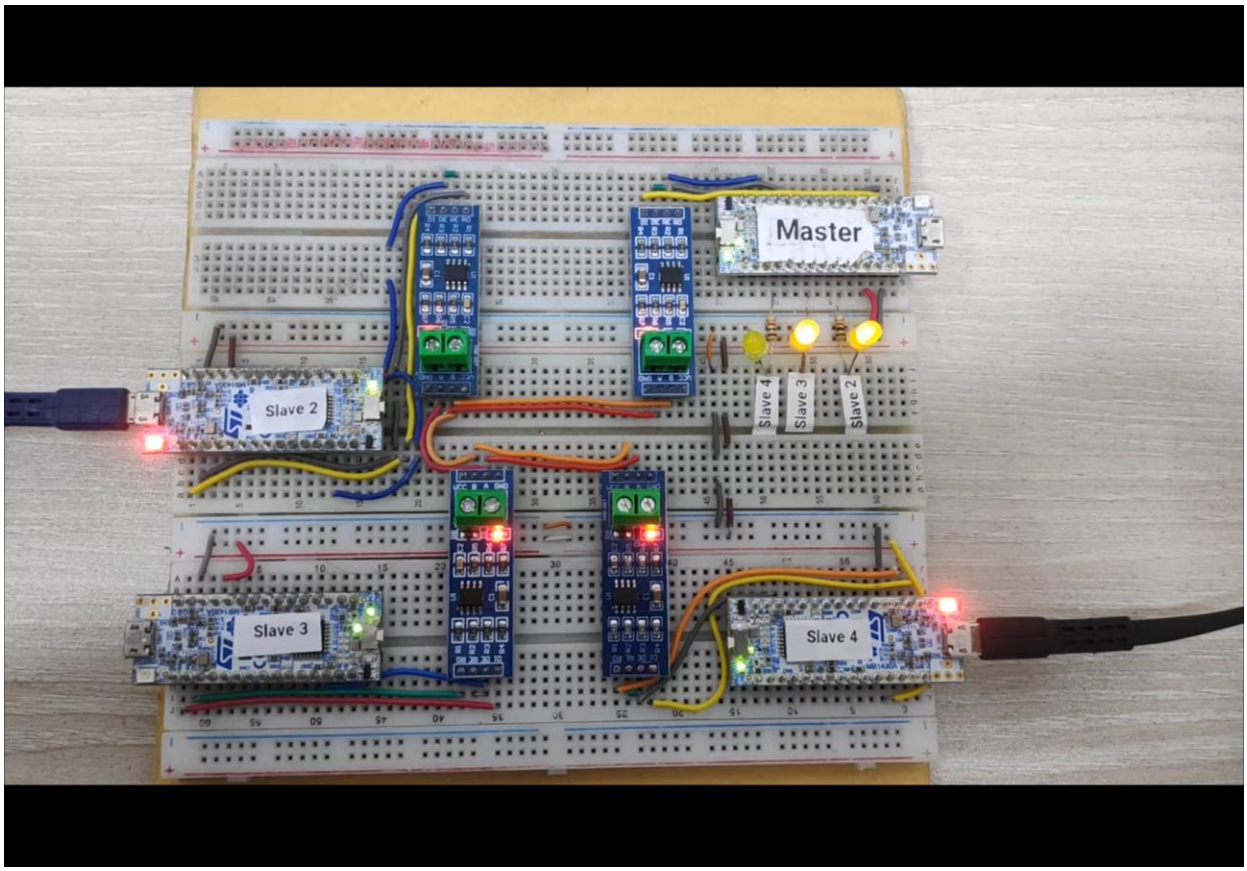

Figure 7. Breadboard prototype of the RS485 master/slave network used to validate the command-and-feedback protocol, with one master and four slave nodes.

Figure 8. PTZ camera assembly mounted on its pan-tilt mechanism, driven by a dedicated slave node and controlled over the TCP/IP network.

6. Testing and Evaluation

6.1. Current System Testing

The test program defined in Section 4 was exercised on the integrated platform and on the breadboard prototype of Figure 7. RS485 signal-integrity and throughput tests (Tests 1 and 2) were conducted on the master/slave network; TCP/IP stability, latency, and video tests (Tests 3–5) were conducted over the W5500 link to the client; and the integration tests (Tests 6 and 7) exercised both networks together under live operation of the drive system, the 6DOF arm, and the PTZ cameras.

Qualitatively, the client software maintained a continuous, structured telemetry view of every subsystem throughout operation, as shown in Figures 5 and 6, and the OFFLINE indication correctly reflected a disconnected node. The heartbeat counters incremented as expected, confirming that the feedback path from each slave to the master to the client remained live.

Note on quantitative results. This draft does not yet report measured figures for latency, round-trip time, throughput, packet-loss counts, or the 24-hour stability run. These measurements are to be inserted here once the logged data is consolidated; the corresponding tables and plots are placeholders pending that data.

6.2. Evaluation

The results to date indicate that the dual-network architecture meets its core design goal: internal coordination and external control proceed concurrently without the two traffic classes interfering, and the feedback layer gives the operator a faithful, continuously updated view of the machine. This observability is the most directly autonomy-relevant outcome, because an autonomous controller would rely on exactly the same telemetry stream to make decisions.

Several limitations bound these conclusions. The evaluation is at present largely qualitative and awaits the quantitative measurements noted above. The current system is teleoperated, so no autonomous decision-making has been exercised end to end. And robustness under sustained field conditions — vibration, temperature, and intermittent wireless links — has not yet been characterised. These limitations define the immediate next steps rather than undermining the architecture, whose separation of concerns is independent of them.

7. Potential for Autonomous Development

7.1. Future Directions

The communication and feedback layer described here is deliberately designed to be the substrate for autonomy rather than autonomy itself. The natural next step is to introduce a perception and decision-making layer that consumes the existing telemetry stream and emits the existing command set. Concretely, this could mean adding environmental and proprioceptive sensors (for example inertial, range, and vision sensors) whose data is carried over the same networks; inserting an onboard or edge compute element that subscribes to the master's telemetry and issues commands at the same TCP/IP boundary the human client uses today; and layering control algorithms — from closed-loop motion control up to learned policies — that progressively take over functions currently performed by the operator.

Because an autonomous controller would attach at the same interface as the client software, much of the existing system can be reused unchanged. The path to autonomy is therefore additive: the dual-network backbone, the per-subsystem feedback records, and the bridging master node all remain, while a new decision-making component is introduced alongside the human operator and gradually assumes more responsibility.

7.2. Challenges and Solutions

Latency and determinism. Autonomous control loops are more sensitive to timing than a human operator. The RS485 side already provides deterministic, low-latency messaging; the main work is to characterise and bound end-to-end latency through the master and across the wireless link, and to keep time-critical loops on the deterministic side of the architecture.

Sensing and perception. Autonomy requires far richer sensing than teleoperation. The solution is to extend the existing node-and-message pattern to sensor nodes, so that perception data flows through the same observable, extensible fabric rather than a parallel ad hoc path.

Compute and integration. Decision-making algorithms exceed the capacity of the microcontroller nodes. Introducing an edge compute element that attaches at the TCP/IP boundary keeps the existing firmware intact while providing the processing headroom that learning-based or planning-based control requires.

Safety and fallback. An autonomous system must fail safe. The existing heartbeat and OFFLINE mechanisms provide a starting point for fault detection, and the retained human-client interface provides a natural manual-override and supervisory path during the transition to autonomy.

8. Conclusion

8.1. Summary

This paper described a dual-network feedback communication system for a multi-node, remotely operated robot. By assigning deterministic internal coordination to an RS485 bus and scalable external control to a TCP/IP link bridged through a single master node, the architecture satisfies two communication requirements that are difficult to meet together, while keeping the design implementable on low-cost microcontrollers. A bidirectional feedback path returns structured telemetry from every subsystem to a remote client, giving the operator continuous observability of the drive system, the 6DOF manipulator, and the PTZ cameras. A structured test program covering signal integrity, throughput, connection stability, latency, video, and combined-network operation was defined and exercised, with quantitative measurements to follow.

8.2. Vision for Future Development

The current platform is teleoperated, but its value lies in what it makes possible. The reliable, observable, dual-network control-and-feedback layer is precisely the groundwork on which autonomy can be built: an autonomous controller would consume the same telemetry and drive the same commands that the system already supports today. With the addition of richer sensing, an edge compute element at the existing network boundary, and progressively more capable control algorithms, the platform can evolve from remote operation toward genuine autonomy — an evolution for which the architecture presented here is the deliberate foundation.